

VANI SURUVU

Apache Spark DAG Features

Directed Acyclic Graph - Brain of Spark Job



In Spark, it's not just code that flows — it's logic, speed, and resilience powered by the DAG.

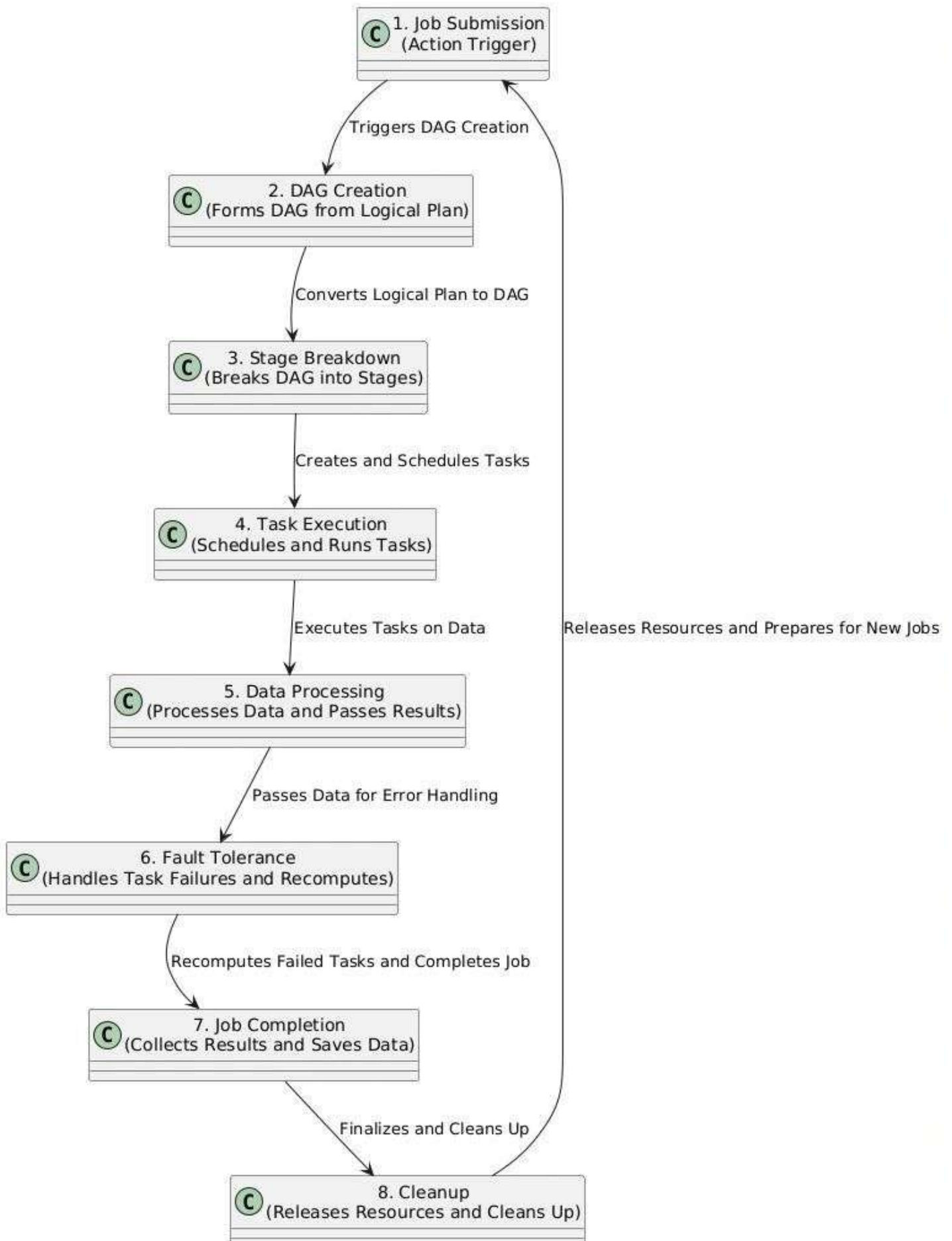


DATA ENGINEER

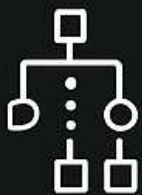
Spark DAG Operation

DAG (Directed Acyclic Graph)

C A DAG is a directed graph with no cycles. It represents a sequence of stages and tasks where each stage depends on the previous one.



DAG in Apache **Spark**



Stage Pipelining

Multiple transformations can be grouped into a single stage for optimized execution



Fault Isolation

Only failed tasks/stages are recomputed, not the entire job



Lineage Tracking

Tracks every transformation for fault recovery without data duplication



Parallel Task Scheduling

Tasks run concurrently across the cluster for maximum speed



Deterministic Execution

Guarantees consistent results for the same input every time



Spark UI Visualization

DAGs can be tracked visually in the Spark UI for debugging and insights



Logical vs Physical Plan Separation

DAG is optimized from a logical plan before execution begins



Resource Optimization

Smart caching and reuse of computations for better cluster utilization

Advanced Features of DAG in Spark

1 Stage Pipelining

- Spark can **pipeline transformations** within the same stage to reduce execution time.
- Example: `map → filter → flatMap` can be executed together without shuffle.

2 Lineage Tracking

- Every RDD or DataFrame transformation maintains a **lineage graph**.
- Spark uses this to **recompute lost partitions** in case of failure—**no data duplication required**.

3 Deterministic Computation

- DAG ensures **determinism** in execution: the same inputs and transformations will **always** yield the same results.

4 Logical vs Physical Plan Separation

- DAG is derived from Spark's **logical plan**, which Spark then **optimizes** into a **physical plan** for execution.
- This allows **query optimization** (via Catalyst Optimizer) before execution starts.

5 Lazy Evaluation

- Transformations are not executed immediately—only **actions** trigger the DAG.
- This **reduces unnecessary computation** and allows Spark to optimize the entire plan first.

6 Fault Isolation

- In case of task failure, **only the dependent stage** is recomputed—not the entire pipeline.
- This minimizes recovery time and saves cluster resources.

7 Parallel Task Scheduling

- DAG allows **tasks within a stage** to run **in parallel** across multiple executors.
- This ensures **maximum resource utilization** and **faster job completion**.

8 Spark UI Visualization

- DAG is **visually available** in the Spark UI under the "Stages" tab.
- Developers can **track job execution**, bottlenecks, and failed stages for deep debugging.

9 Resource Optimization

- Spark uses the DAG to determine **where and when to cache** intermediate results or reuse computations.
- Helps avoid recomputation in iterative algorithms (like ML or GraphX).

10 Support for Complex Workflows

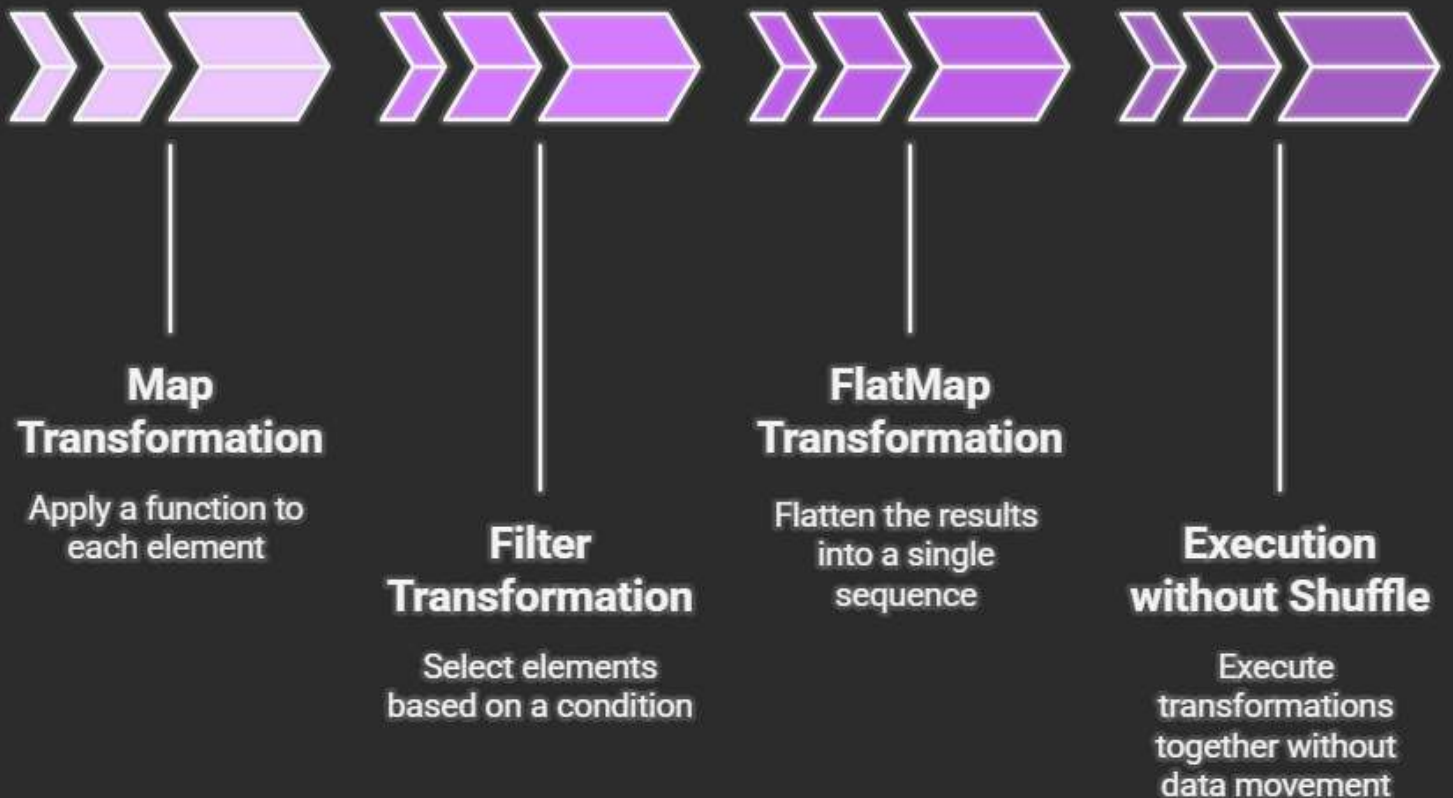
- DAG supports **complex dependencies**, allowing Spark to handle jobs with **multiple branches, joins, co-groups**, and more with efficiency.

🔍 Advanced Features of DAG in Spark

1 Stage Pipelining

- Spark can **pipeline transformations** within the same stage to reduce execution time.
- Example: `map` → `filter` → `flatMap` can be executed together without shuffle.

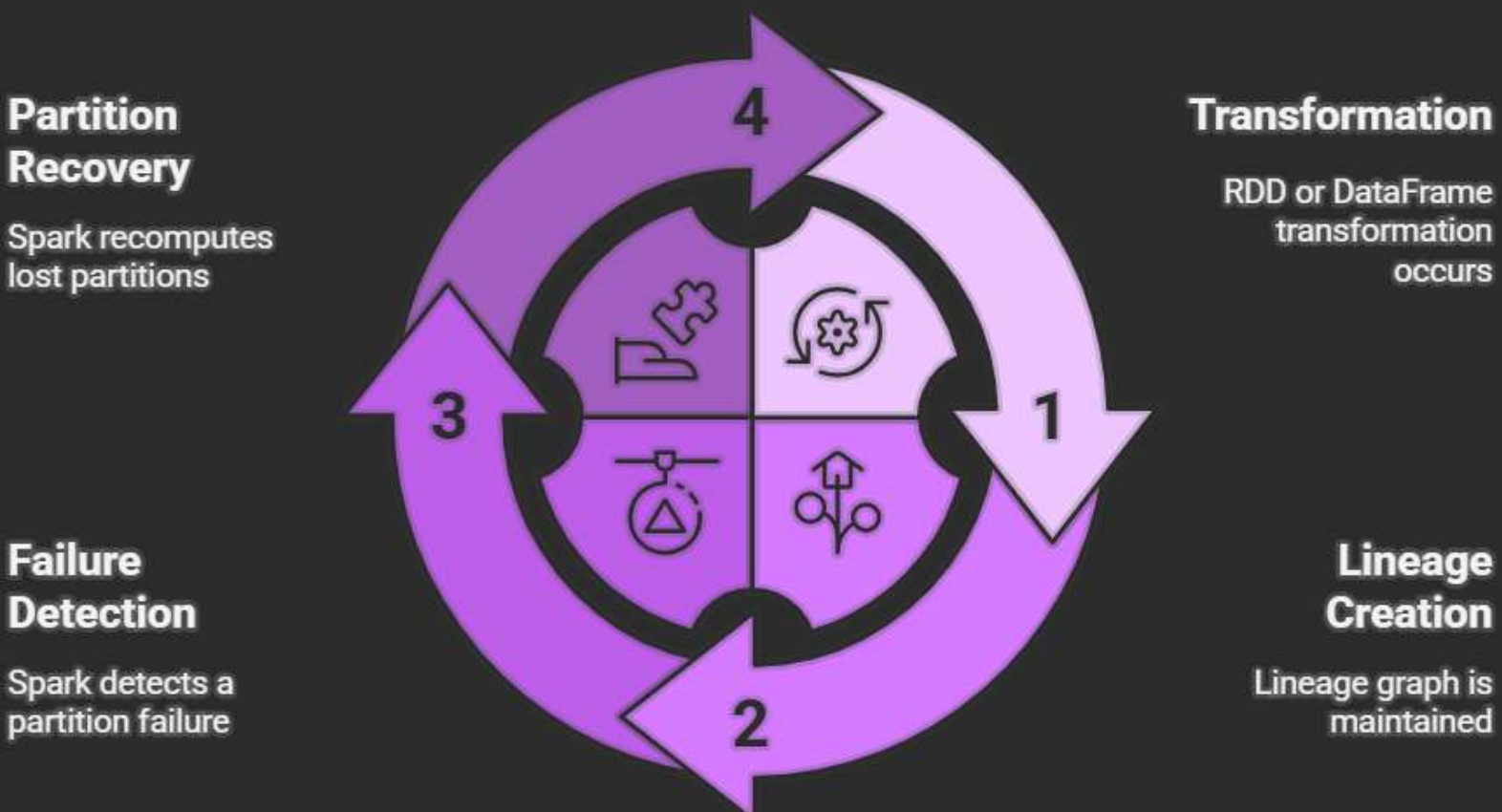
Spark Transformation Pipelining



2 Lineage Tracking

- Every RDD or DataFrame transformation maintains a **lineage graph**.
- Spark uses this to **recompute lost partitions** in case of failure—**no data duplication required**.

Spark's Data Recovery Cycle



3 Deterministic Computation

- DAG ensures **determinism** in execution: the same inputs and transformations will **always** yield the same results.

DAG Deterministic Execution Cycle



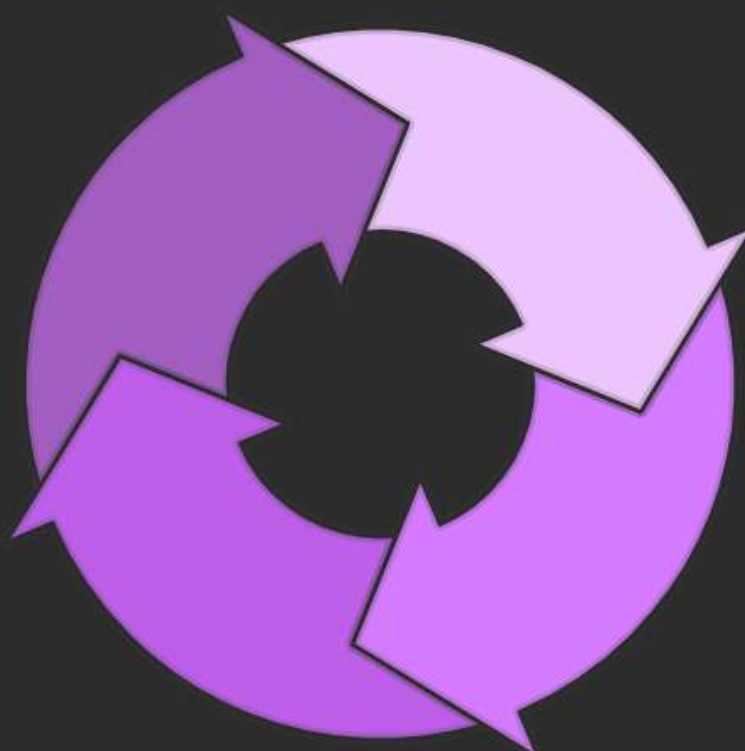
Verify Determinism

Results are checked for consistency



Generate Results

Results are generated from the transformations



Provide Inputs

Inputs are fed into the DAG



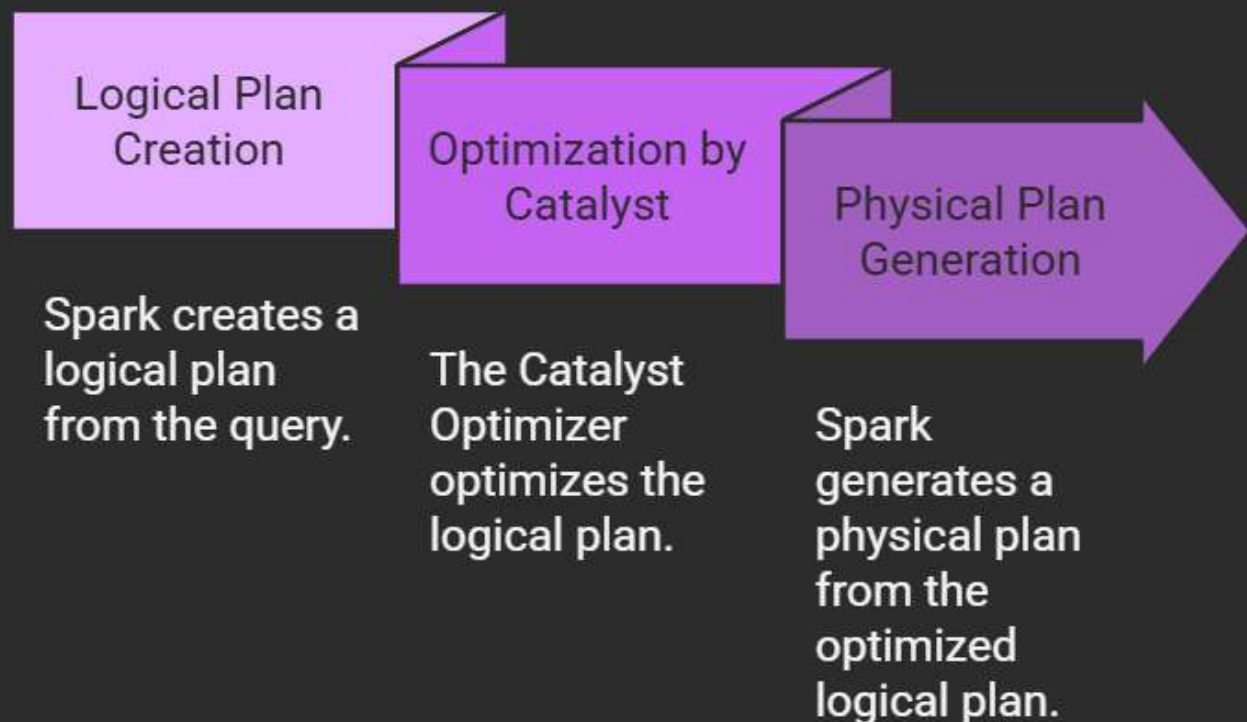
Apply Transformations

Transformations are applied to the inputs

4 Logical vs Physical Plan Separation

- DAG is derived from Spark's **logical plan**, which Spark then **optimizes** into a **physical plan** for execution.
- This allows **query optimization** [via Catalyst Optimizer] before execution starts.

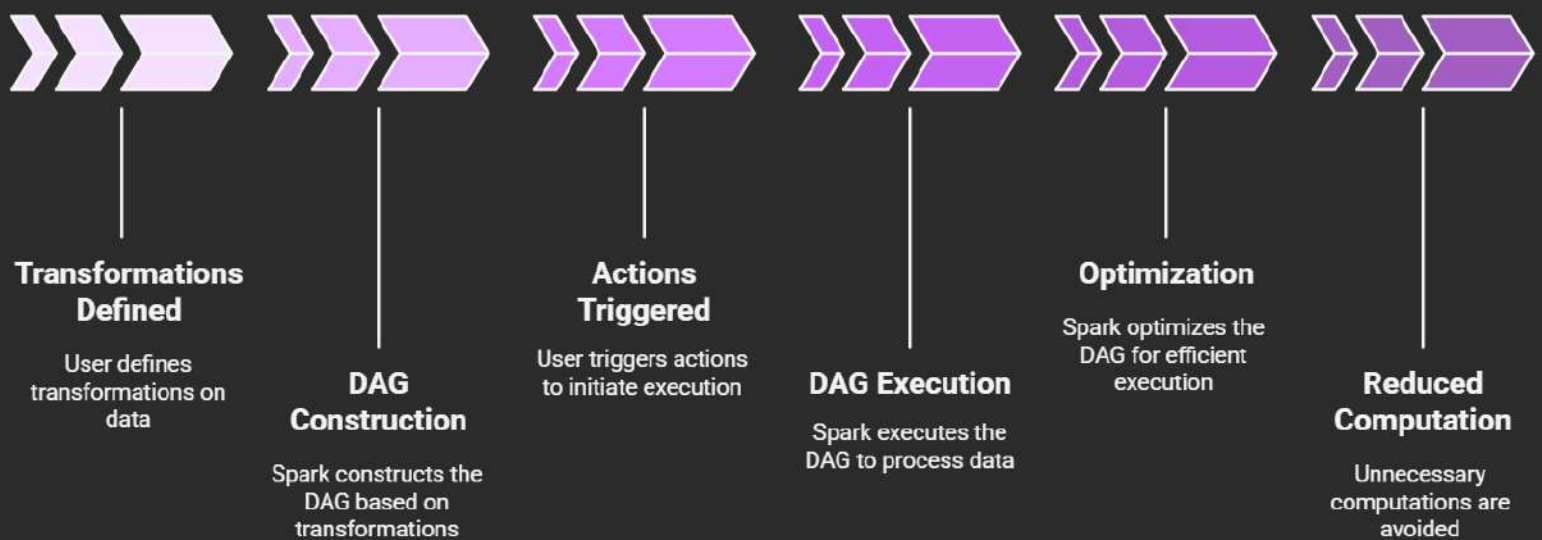
Spark DAG Optimization Process



5 Lazy Evaluation

- Transformations are not executed immediately—only **actions** trigger the DAG.
- This **reduces unnecessary computation** and allows Spark to optimize the entire plan first.

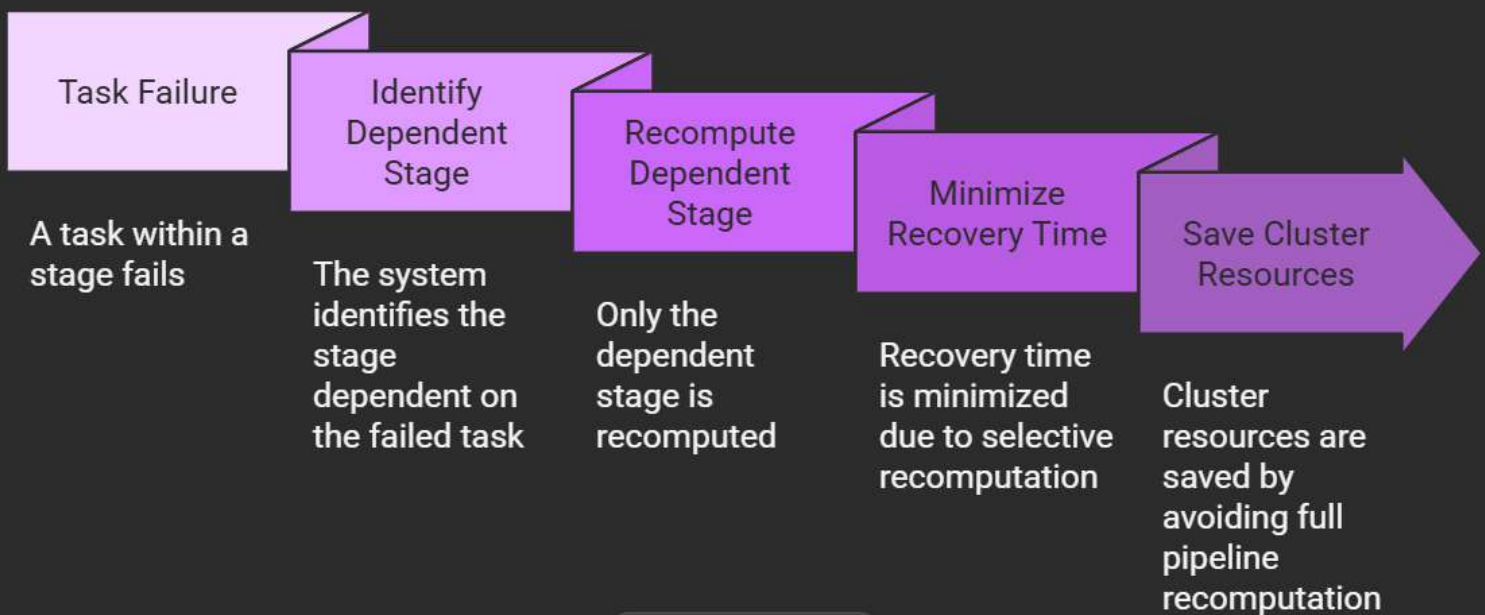
Spark DAG Execution Sequence



6 Fault Isolation

- In case of task failure, **only the dependent stage** is recomputed—not the entire pipeline.
- This minimizes recovery time and saves cluster resources.

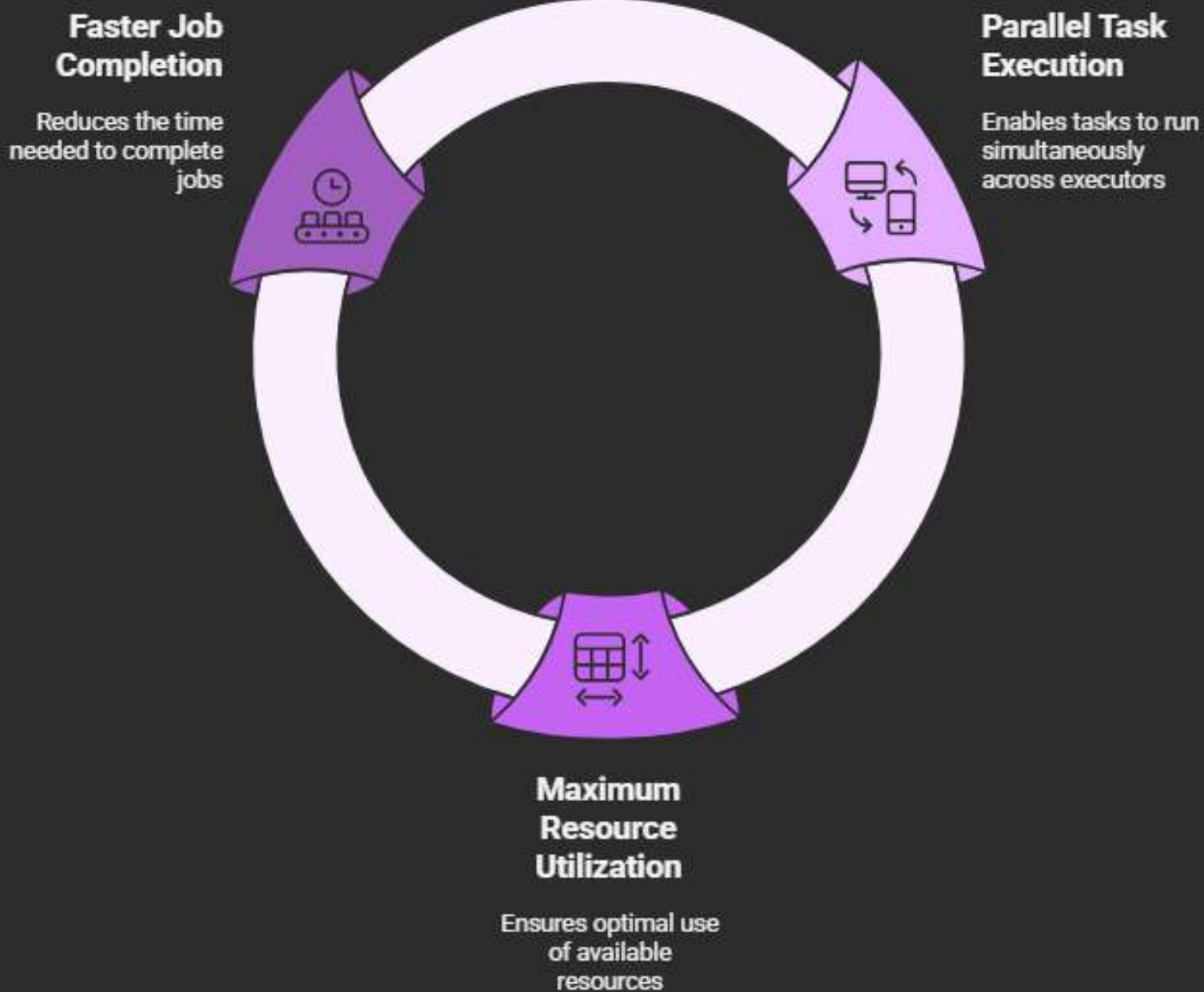
Task Failure Recovery in Spark



7 Parallel Task Scheduling

- DAG allows **tasks within a stage** to run **in parallel** across multiple executors.
- This ensures **maximum resource utilization** and **faster job completion**.

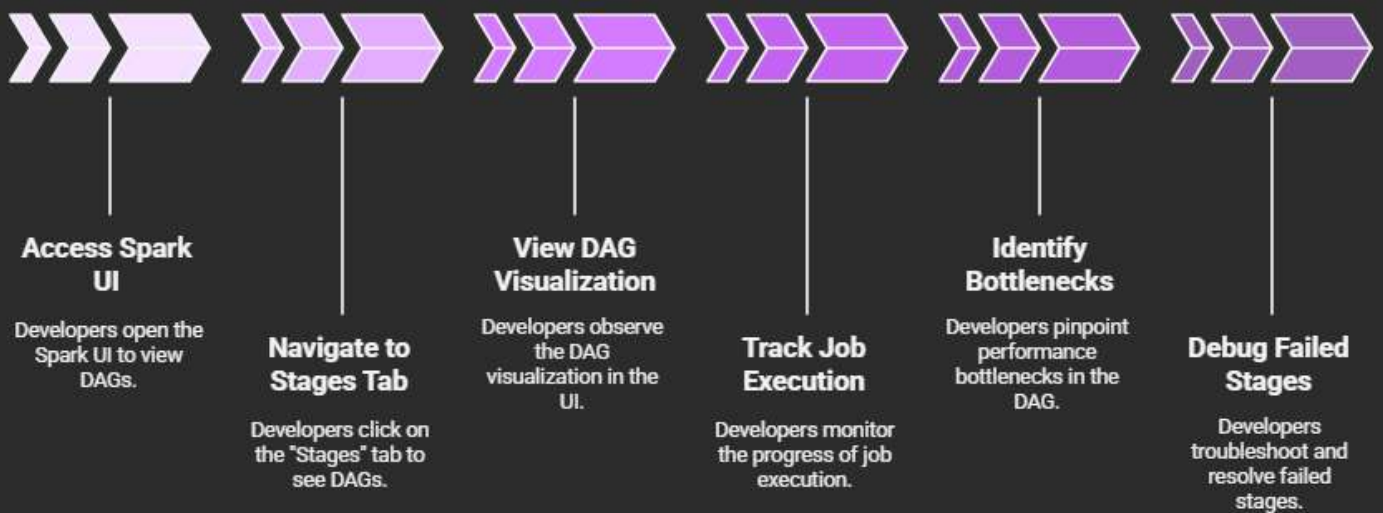
Enhancing Spark Performance with DAG



8 Spark UI Visualization

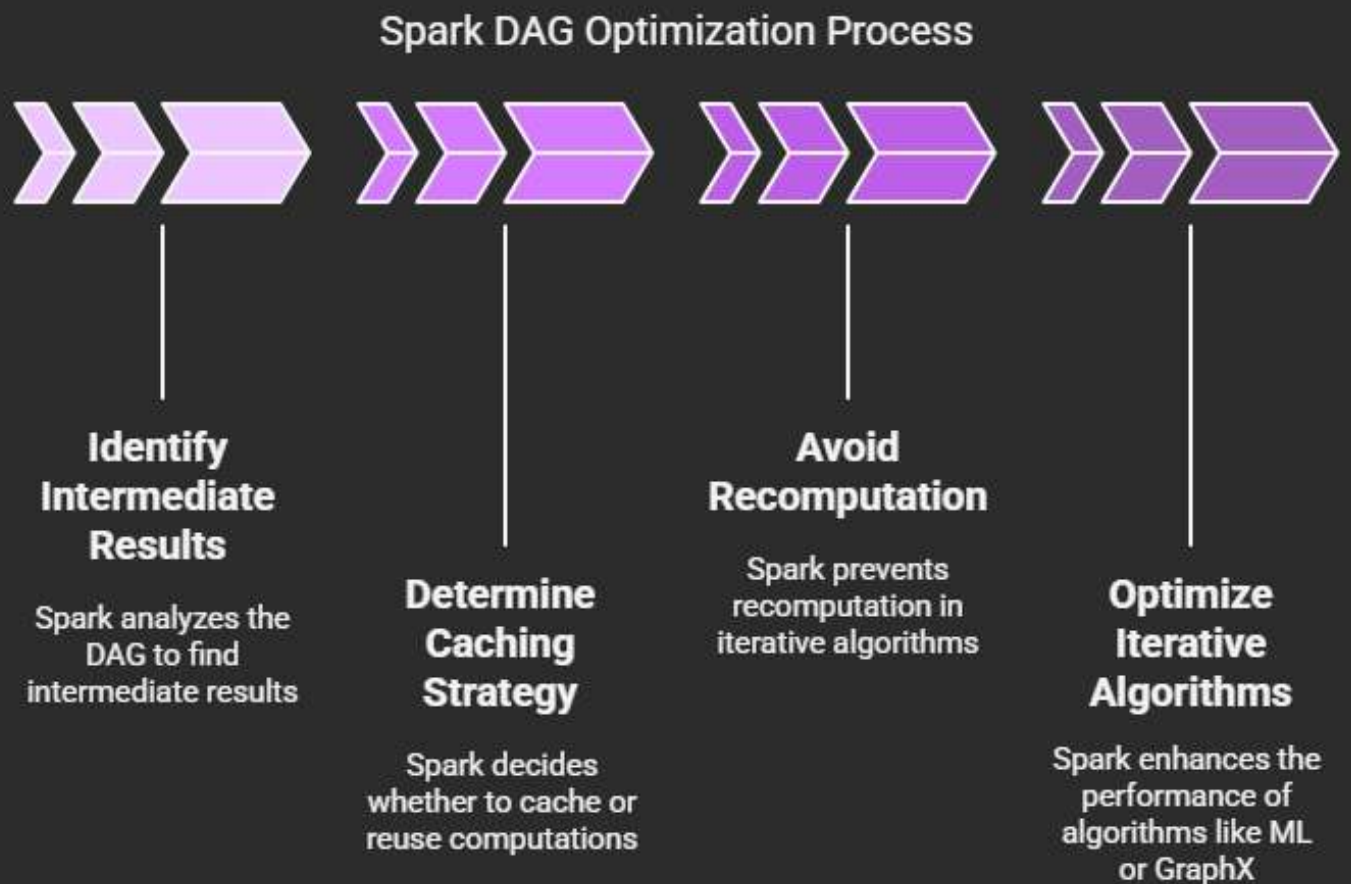
- DAG is **visually available** in the Spark UI under the "Stages" tab.
- Developers can **track job execution**, bottlenecks, and failed stages for deep debugging.

Spark UI DAG Visualization and Debugging



9 Resource Optimization

- Spark uses the DAG to determine **where and when to cache** intermediate results or reuse computations.
- Helps avoid recomputation in iterative algorithms [like ML or GraphX].



10 Support for Complex Workflows

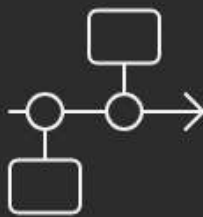
- DAG supports **complex dependencies**, allowing Spark to handle jobs with **multiple branches, joins, co-groups**, and more with efficiency.

DAG supported operations



Complex dependencies

DAG supports complex dependencies between operations.



Multiple branches

DAG allows multiple branches within the job.



Joins

DAG supports join operations for combining data.



Co-groups

DAG supports co-group operations efficiently.